Question No. 1 CLO 09, SO 1 (4 Points)

Part 1: Choose **False** or **True** for each of the following statements (2 Points):

1) ( ) Boolean is a primitive data type for the c programming language. (T/F)

2) ( ) ** operator in Fortran and MATLAB is left associative. (T/F)

3) ( ) Jagged Array is a multi-dimensioned array in which all rows have a varying number of elements. (T/F)

4) ( ) Type checking is done statically in both Python and JavaScript programming languages. (T/F)

Part 2: Choose the **correct answer** for each of the following statements (2 Points):

1) What is a potential issue associated with operator overloading?
   a. Loss of compiler error detection.
   b. Increased readability of code.
   c. Improved performance of programs.
   d. Better code organization.

2) Which of the following expressions may lead to a problem due to short-circuit evaluation?
   a. (a < b) && (b < 10)
   b. (a > b) || (b > 0)
   c. (a == b) || (a = 0)
   d. (a != b) && (a > 0)

3) After loop termination in Ada's for statement, what happens to the loop variable?
   a. It retains its last assigned value.
   b. It becomes undefined.
   c. It becomes null.
   d. It is implicitly undeclared.

4) Which of the following is an ordinal data type?
   a. Enumeration
   b. Float
   c. Record
   d. String

Question No. 1 CLO 09, SO 1 (4 Points)

Choose the **correct answer** for each of the following statements:

1) In the context of formal language definition, what is a "lexeme"?
   a) A string of characters over some alphabet.
   b) The syntax analysis part of a compiler.
   c) The lowest-level syntactic unit of a language (e.g., *, sum, begin).
   d) A category of lexemes (e.g., identifier).

2) What is the primary purpose of a "recognizer" in formal language theory?
   a) To generate sentences of a language.
   b) To determine if the syntax of a particular sentence is correct.
   c) To read input strings and decide whether they belong to the language.
   d) To translate code from one language to another.

3) What does it mean for a programming language to be "strongly typed"?
   a) That all variables must be explicitly declared with a type.
   b) Those type errors are always detected.
   c) That implicit type conversions are allowed.
   d) That the language uses dynamic type binding.

4) What is a "narrowing conversion"?
   a) Coercion
   b) Type Casting
   c) Converting an integer to a floating-point number.
   d) Converting a struct to a union.

Question No. 2

Show the outputs of the following code in the following 2 scopes:

```
#include <stdio.h>
int a = 10, b = 20;
void Alpha() {
a += 2;
b += 3;
printf("Alpha: a = %d, b = %d\n", a, b);
}
void Beta() {
int b = 5;
a += 1;
b += 2;
printf("Beta: a = %d, b = %d\n", a, b);
}
void main() {
int a = 1, b = 2;
Alpha();
Beta();
}
```

Static scope	Dynamic scope

Question No. 3 HVCLO 13, SO 1 (8 Points)

Find the value of the following expressions

<table>
<tr><th>#</th><th>Language</th><th>Expression</th><th>Value</th></tr>
<tr><td>1.</td><td>Ada</td><td><code>2**3**0</code></td><td></td></tr>
<tr><td>2.</td><td>C language</td><td><code>char * s= "AAA\0BBB";</code>
<code>printf("%s",s);</code></td><td></td></tr>
<tr><td>3.</td><td>C language</td><td><code>int b=5;</code>
<code>int a= (b || ++b>5);</code></td><td></td></tr>
<tr><td>4.</td><td>C language</td><td><code>int b=5;</code>
<code>int a= (b && b++>30);</code></td><td></td></tr>
<tr><td>5.</td><td>C language</td><td><code>#include <stdio.h></code>
<code>union Data{</code>
<code> int y;</code>
<code> int x;</code>
<code>};</code>
<code>int main() {</code>
<code> union Data data;</code>
<code> data.y = 15;</code>
<code> printf(" %d\n", data.x);</code>
<code> return 0;</code>
<code>}</code></td><td></td></tr>
<tr><td>6.</td><td>C language</td><td><code>int x=15;</code>
<code>int y = (x=5)?3:4;</code></td><td></td></tr>
<tr><td>7.</td><td>C language</td><td><code>while(-1)</code>
<code> printf("Cpcs301\n");</code>
<code>printf("OK!");</code></td><td></td></tr>
<tr><td>8.</td><td>C language</td><td><code>int a= !(2<5<3) ;</code></td><td></td></tr>
<tr><td>9.</td><td>Perl</td><td><code>-7%4</code></td><td></td></tr>
<tr><td>10.</td><td>C sharp</td><td><code>int x=7; int y=5;</code>
<code>(x>3)?y :x=20 ;</code></td><td></td></tr>
<tr><td>11.</td><td>C++</td><td><code>int(3.2);</code></td><td></td></tr>
<tr><td>12.</td><td>Java</td><td><code>int a=0 ;</code>
<code>int x=(15)*a*(19/4*8) ;</code></td><td></td></tr>
<tr><td>13.</td><td>Java</td><td><code>x=y=z=w=50;</code>
<code>system.out.println(z);</code></td><td></td></tr>
<tr><td>14.</td><td>Pascal</td><td><code>x : boolean ;</code>
<code>x := 12>7 ;</code></td><td></td></tr>
<tr><td>15.</td><td>Lisp</td><td><code>(+ (* 4 5) 6)</code></td><td></td></tr>
<tr><td>16.</td><td>Python</td><td><code>A= {1:2, 2:3, 4:9}</code>
<code>print(A[2])</code></td><td></td></tr>
</table>

Question No. 4 CLO 7, SO 1 (4 Points)

Consider the following BNF grammar for simple arithmetic expressions.

```
<expression> ::= <term> OR <term> | <term>
<term> ::= <factor> AND <factor> | <factor> | <comparison>
<factor> ::= NOT <factor> | <bool> | <expression>
<comparison> ::= <var> <comp_op> <number>
<comp_op> ::= '==' | '!=' | '<' | '>'
<var> ::= 'x' | 'y' | 'z'
<number> ::= '1' | '2' | '3'
<bool> ::= 'TRUE' | 'FALSE'
```

a. Determine all terminal and non-terminal symbols **(2 points)**

Terminal =

Non-terminal =

b. Provide an example of a logical expression that demonstrates the ambiguity of this grammar (give only the expression, don't draw the parse tree) **(2 points)**

Question No. 5 CLO14, SO 1 (5 Points)

A. Trace and Interpret Regular Expression Matches: (4 Points)

```
import re

sentence = "GPT3, BERT, and T5 are advanced NLP models from OpenAI and Google."
match1 = re.findall(r'\b[A-Z]{2,}[0-9]\b', sentence)
match2 = re.findall(r'[A-Z][a-z]+', sentence)
match3 = re.findall(r'[A-Z].+?[a-z]', sentence)
match4 = re.findall(r'[A-Z][A-Z0-9]+', sentence)

print("match1 =", match1)
print("match2 =", match2)
print("match3 =", match3)
print("match4 =", match4)
```

Fill in the expected output for each of the following variables:

- match1 =
- match2 =
- match3 =
- match4 =

B. Write a Regular Expression that accepts only even number of binary code (e.g. 01, 1101, 00, 0001, 11001100, 0101 etc.) (1 Point)

Question No. 6 CLO15, SO 1 (5 Points)

a) Determine the output of the following instruction loops (0.5 point for each) **(3 Points)**

languages	Instructions	Outputs

python	`R = [i for i in range(10) if i % 3 == 0]`
	`S = [(x, x**2) for x in R if x % 2 == 0]`
	`print(S)`	

python	`K = {i * j for i in range(1, 4) for j in range(2)}` `print(K)`	

C Language	`int x= 1;`
	`while(x=1){`
	`    printf("x=%d\n",x++);`
	`}`
	`printf("x=%d\n",x);`	

C Language	`int X=5;`
	`if(X%2)`
	`    printf("Even");`
	`else`
	`    printf("Odd");`	

C++	`for(int x=1;x<=2;x++)` `    printf("x=%d\n",x);`	
C Language	`for(;-3;)` `    printf("Hi!");`	

b) Write the operational semantics descriptions for the following C code: **(2 Points)**

```
int j=0, i=0;
for (j=0,i=0 ; j<=5 || i<3 ; j++,i+=2) {
    if (j == 3) continue;
    if (j == 4) break;
    j+=2;
}
```

Question No. 7 CLO15, SO 1 (4 Points)

Determine the output of the following instruction loops (1 point for each)

languages	**Instructions**	**Outputs**

R	`X <- c(3,2,1,6,4,3)`
	`dim(X) <- c(2,3)`
	`apply(X,2,function(x) 2*max(x)+1)`	

C language	`int x = 2;`
	`switch (x % 2) {`
	`    case 0: x += 1;`
	`    case 1: x += 2;`
	`}`
	`printf("%d", x);`	

Python	`S=[x**2 for x in range(5)]`
	`T= [(x, x*4) for x in S if x % 2 ==0 ]`
	`print (T)`	

Python	`A={2*x+y for x in range(3) for y in range(2)}` `print (A)`	

Good Luck

Question No. 4 CLO16, SO 1 (5 Points)

A. Consider the following python payroll calculator:

```
def fun1(a, b, c=0, d=1):
print(a- b + c* d)
```

Fix the incorrect function calls below. (1 for each row)

Function call	Output
fun1(2,4)	
fun1(4,d=2)	
fun1(a=1,c=2,b=3)	
fun1(a=1,c=4,b=4,d=2)	
fun1(2,c=2,b=0)	